Lesson 4 Worksheet: The Manhattan Algorithm — ANSWER KEY

Part 1: Manhattan Distance

Calculate the Manhattan distance for each pair. Remember: Manhattan distance = |row difference| + |column difference|.

Start	Destination	Row Distance	Col Distance	Manhattan Distance
(0, 0)	(2, 3)	|2 - 0| = 2	|3 - 0| = 3	5
(3, 3)	(0, 0)	|0 - 3| = 3	|0 - 3| = 3	6
(1, 0)	(1, 3)	|1 - 1| = 0	|3 - 0| = 3	3
(0, 2)	(3, 2)	|3 - 0| = 3	|2 - 2| = 0	3
(2, 1)	(2, 1)	|2 - 2| = 0	|1 - 1| = 0	0

Which pair requires the most steps? (3, 3) to (0, 0) with a Manhattan distance of 6.

Which pair requires zero steps? Why? (2, 1) to (2, 1) — the start and destination are the same position, so no movement is needed.

Part 2: Write compute_path — Positive Directions Only

```
def compute_path(start, end):
    path = []
    current_row, current_col = start
    dest_row, dest_col = end

    # Move south (rows increase)
    while current_row < dest_row:
        current_row = current_row + 1
        path.append((current_row, current_col))

    # Move east (columns increase)
    while current_col < dest_col:
```

```
        current_col = current_col + 1
        path.append((current_row, current_col))

    return path
```

Blanks in order: `<` , `1` , `current_row` , `current_col` , `<` , `1` , `current_row` , `current_col`

Test it: `compute_path((0, 0), (2, 3))` *should return:*

[(1, 0), (2, 0), (2, 1), (2, 2), (2, 3)]

How many steps? `len(path)` *= 5*

Try this: `compute_path((3, 3), (1, 0))` *— what does it return?*

[] (empty list)

Why is the result wrong? Because 3 is not less than 1 and 3 is not less than 0, neither while loop condition is true, so neither loop executes. The function only handles positive directions (south and east) but this path requires north and west movement.

Part 3: Write compute_path — All Directions

```
def compute_path(start, end):
    path = []
    current_row, current_col = start
    dest_row, dest_col = end

    # Move south (rows increase)
    while current_row < dest_row:
        current_row = current_row + 1
        path.append((current_row, current_col))

    # Move north (rows decrease)
    while current_row > dest_row:
        current_row = current_row - 1
        path.append((current_row, current_col))

    # Move east (columns increase)
    while current_col < dest_col:
        current_col = current_col + 1
        path.append((current_row, current_col))

    # Move west (columns decrease)
    while current_col > dest_col:
        current_col = current_col - 1
        path.append((current_row, current_col))
```

```
    return path
```

Blanks in order: `>` , `-` , `1` , `append` , `current_row` , `current_col` , `>` , `-` , `1` , `append` , `current_row` , `current_col`

Part 4: Hand-Trace the Algorithm

Path 1: compute_path((0, 0), (2, 3))

Row moves:

Step	Action	Position Added
1	Row +1	(1, 0)
2	Row +1	(2, 0)

Column moves:

Step	Action	Position Added
3	Col +1	(2, 1)
4	Col +1	(2, 2)
5	Col +1	(2, 3)

Returned path: [(1, 0), (2, 0), (2, 1), (2, 2), (2, 3)]

len(path): 5 *Manhattan distance:* 5 *Do they match?* YES

Path 2: compute_path((3, 3), (1, 0))

Row moves:

Step	Action	Position Added
1	Row -1	(2, 3)
2	Row -1	(1, 3)

Column moves:

Step	Action	Position Added
3	Col -1	(1, 2)

Step	Action	Position Added
4	Col -1	(1, 1)
5	Col -1	(1, 0)

Returned path: [(2, 3), (1, 3), (1, 2), (1, 1), (1, 0)]

len(path): 5 **Manhattan distance:** 5 **Do they match?** YES

Path 3: compute_path((2, 0), (2, 3)) (same row!)

Do any row loops execute? NO **Why?** current_row (2) is not less than dest_row (2) and not greater than dest_row (2), so both row loops are skipped.

Column moves:

Step	Action	Position Added
1	Col +1	(2, 1)
2	Col +1	(2, 2)
3	Col +1	(2, 3)

Returned path: [(2, 1), (2, 2), (2, 3)]

len(path): 3 **Manhattan distance:** 3 **Do they match?** YES

Path 4: compute_path((1, 1), (1, 1)) (same position!)

Do any loops execute? NO **Why?** All four while loop conditions are false because current_row equals dest_row and current_col equals dest_col.

Returned path: []

len(path): 0

Part 5: Draw the Paths on the Grid

```
          Col 0       Col 1       Col 2       Col 3
           |           |           |           |
Row 0 ---[0,0]------[0,1]------[0,2]------[0,3]---
           |                                   |
           1↓                                  |
Row 1 ---[1,0]------[1,1]------[1,2]------[1,3]---
           |          (4)        2←          2← 2↑
```

```
          1↓                                    |
Row 2 ---[2,0]------[2,1]------[2,2]------[2,3]---
          1→→→→→→→→→3→→→→→→→→→→3→→→→→→→→→→1,3  2↑
                                                  |
Row 3 ---[3,0]------[3,1]------[3,2]------[3,3]---
```

What shape do most of the paths make? An L-shape (right angle). The algorithm moves along rows first, then turns and moves along columns.

Which paths are straight lines instead of L-shapes? Path 3 (moves only along columns because same row). Path 4 is a single point (no movement).

Part 6: Verify Your Understanding

1. *The Manhattan algorithm always moves rows first, then columns.*
2. *If the destination row is SMALLER than the start row, which while loop runs?* The north loop (`while current_row > dest_row`).
3. *If the destination column is LARGER than the start column, which while loop runs?* The east loop (`while current_col < dest_col`).
4. *When the start and destination are in the same row, the row loops do not execute.*
5. *When the start and destination are the same position, the path is [] (empty list) and len(path) is 0.*
6. *The number of steps in a path (len(path)) always equals the Manhattan distance.*
7. *Why don't we need if/else to choose direction? How do the 4 while loops handle it?*

   Each while loop has a condition that is only true for one direction. If the robot needs to go south, only the south loop's condition (`current_row < dest_row`) is true. The north loop's condition (`current_row > dest_row`) is false, so it is automatically skipped. The same applies for east/west. Only the loops that need to run actually execute.

Reflection

You wrote a function that can find a path between ANY two grid positions using just 4 while loops. In the next lesson, you will wrap this function into a class. Why is it important to get the function working correctly BEFORE putting it in a class?

Getting the function working first lets you focus on the algorithm logic without worrying about class syntax (`self` , `__init__` , etc.). You can test the function directly with simple calls like `compute_path((0,0), (2,3))` and verify the output. Once you know the algorithm is correct, wrapping it in a class is a small, manageable step. Trying to learn both the algorithm and class syntax at the same time makes it harder to find bugs — you wouldn't know if a problem was in the algorithm or the class structure.

Next Lesson: We'll wrap `compute_path` *into a Manhattan class that remembers the robot's position!*